
THEORY OF COMPUTATION NOTES

SAYAN KAR

Last updated

August 18, 2026

Abstract

Personal notes on Theory of Computation, following Kozen's *Automata and Computability*. Chapter 1 covers finite automata and regular languages, Chapter 2 covers context-free grammars and pushdown automata, Chapter 3 covers Turing machines and effective computability, and Chapter 4 collects selected solutions to the miscellaneous exercises.

I would also like to thank Claude (Sonnet 5) for typesetting these notes in $\text{\LaTeX}$ from my handwritten pages, using my own template. The template is available at: <https://github.com/karsayan2357/my-templates/tree/main/notes/notes>.

NOTATIONS

$\mathbb{F}_q$	Finite field of size q
$\mathbb{F}_q^*$	Multiplicative group of $\mathbb{F}_q$
$\mathbb{Z}_m$	Residue classes modulo m
$\mathbb{N}$	Set of natural numbers
$[n]$	The set $\{1, 2, \dots, n\}$
$\emptyset$	Empty set
$X \sim \mathcal{D}$	Random variable X having distribution $\mathcal{D}$
$\text{Unif}(S)$	Uniform distribution over the set S
$\mathbb{E}[X]$	Expectation of the random variable X
$\mathbb{D}[X]$	Variance of the random variable X
$\langle \mathbf{u}, \mathbf{v} \rangle$	Inner product of the vectors $\mathbf{u}$ and $\mathbf{v}$
$A \otimes B$	Kronecker product of A and B
$A \odot B$	Hadamard product of A and B
$\ \mathbf{a}\ $	Hamming weight of $\mathbf{a}$
$\mathcal{O}(\cdot), \tilde{\mathcal{O}}(\cdot)$	Asymptotic upper bound, and the same suppressing logarithmic factors

CONTENTS

Notations	ii
1 Finite Automata and Regular Languages	1
1.1 Finite Automata	1
1.2 Languages	2
1.2.1 Computation of a DFA	4
1.3 Non-Determinism	5
1.3.1 Regular Operations	7
1.4 Nondeterministic Finite Automata	7
1.4.1 Difference between DFA and NFA	8
1.5 Equivalence of NFA and DFA	9
1.6 Closure under Union, Concatenation, and Star	11
1.7 Regular Expressions	12
1.7.1 Properties of RE	13
1.8 Converting a DFA to a Regular Expression	15
1.8.1 A Motivating Example	16
1.9 More Closure Properties of Regular Languages	17
1.10 Reversal of a Language	18
1.11 Homomorphisms	19
1.12 Inverse Homomorphism	19
Non-Regular Languages	20
DFA Minimization	23

2	Context-Free Grammars and Pushdown Automata	26
2.1	Context-Free Grammars	26
2.2	Parse Trees, Derivation & Ambiguity	28
2.3	Normal Forms	30
2.4	Pumping Lemma for CFLs	32
2.5	Proving a Language is not a CFL	32
2.6	Pushdown Automata	33
2.7	NPDA and CFGs are Equivalent	34
2.8	Closure Properties of CFLs	35
3	Turing Machines and Effective Computability	37
4	Selected Solutions to Miscellaneous Problems (Kozen)	38

CHAPTER 1

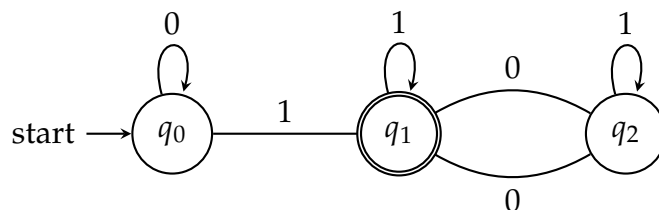
FINITE AUTOMATA AND REGULAR LANGUAGES

1.1 Finite Automata

Definition 1.1 (Deterministic Finite Automaton). A *deterministic finite automaton* (DFA) is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set called *states*,
2. Σ is a finite set called the *alphabet*,
3. $\delta : Q \times \Sigma \rightarrow Q$ is a *transition function*,
4. $q_0 \in Q$ is the *start state*, and
5. $F \subseteq Q$ is the *set of accept states*.

Example 1.2. Look at the state diagram of the following finite automaton M_1 .



Now a finite automaton takes a binary string, such as 1101, as input and gives out the output if the input is accept or reject.

Q. How to quantify this accept/reject?

Reading through the string from left to right, taking in each binary digit, the FA changes its state. If the last state is an accept state, such as q_1 , then the output is accept, otherwise reject.

For this example, on input 1101:

digit	state transition
1	$q_0 \rightarrow q_1$
1	$q_1 \rightarrow q_1$
0	$q_1 \rightarrow q_2$
1	$q_2 \rightarrow q_1$

$\therefore$ 1101 is accepted.

For this example,

(i) $Q = \{q_0, q_1, q_2\}$

(ii) $\Sigma = \{0, 1\}$

(iii) $\delta : Q \times \Sigma \rightarrow Q$ with the following table

δ	0	1
q_0	q_0	q_1
q_1	q_2	q_1
q_2	q_1	q_2

(iv) q_0 is the start state.

(iiv) $F = \{q_1\}$.

1.2 Languages

The set of all strings A that M accepts is called the *language of machine M* , denoted as $L(M)$.

We say M recognizes $A = L(M)$.

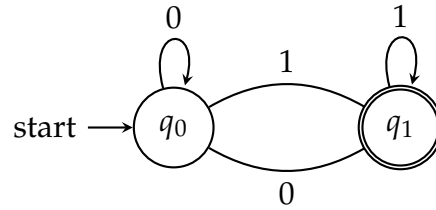
A machine may accept many strings, but it only accepts one language. We also have the empty language ($\emptyset$) for machines accepting no strings.

Example 1.3. For M_1 as before, we have

$$L(M_1) = \{w \in \{0,1\}^* \mid w \text{ has at least one 1 and an even number of 0's follow the last 1}\}.$$

Example 1.4. Here is an example of the state diagram of a finite automaton M_2, M_3, M_4 .

M_2 :



Formally, $M_2 = (\{q_0, q_1\}, \{0, 1\}, \delta, q_0, \{q_1\})$.

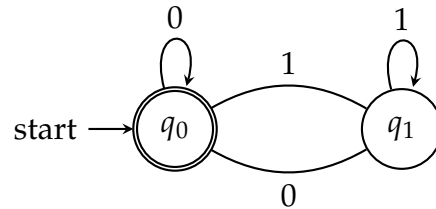
For this example, δ is given by:

δ	0	1
q_0	q_0	q_1
q_1	q_0	q_1

Now, language of M_2 :

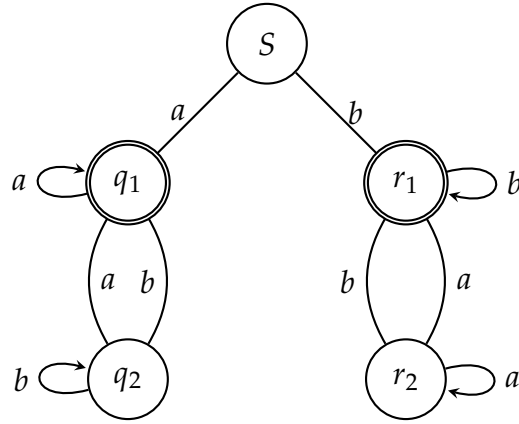
$$L(M_2) = \{w \mid w \text{ ends with 1}\}.$$

M_3 :



$$L(M_3) = \{w \mid w \text{ ends with 0 or } w = \varepsilon\}.$$

M_4 :



$$Q = \{S, q_1, q_2, r_1, r_2\}, \quad \Sigma = \{a, b\}, \quad F = \{q_1, r_1\}, \quad \text{start-state} = S.$$

$$L(M_4) = ?$$

It is easy to see that going from the left component to the right is not possible. And for the left (resp. right) component, the string must start with a (resp. b), and for the string to be accepted, it must end with a (resp. b). Also, we can change to q_1 (resp. r_1) iff the last string letter is a (resp. b). So

$$L(M_4) = \{w \in \{a, b\}^* \mid w \text{ starts and ends with some (the same) letter}\}.$$

Remark 1.5. (1) For a language, there may be different DFAs accepting it (in fact, there can be infinitely many). But each DFA accepts exactly one language.

(2) Given a pair $(\overset{\text{state}}{q}, \overset{\text{symbol}}{a})$, $\exists! q' \in Q$ so that $\delta(q, a) = q'$.

(3) The set of states in a DFA partitions the set of all strings. The set of states is finite.

1.2.1 Computation of a DFA

Let $M = (Q, \Sigma, \delta, q_0, F)$ and $w = a_1 \cdots a_n$ where $a_i \in \Sigma$. We say that M *accepts* w if $\exists$ a sequence of states $r_0, \dots, r_n$ s.t.

$$1) \ r_0 = q_0$$

$$2) \ r_i = \delta(r_{i-1}, a_i) \ \forall i = 1, \dots, n$$

3) $r_n \in F$.

Remark 1.6. The states $\{r_i\}$ need not be distinct.

Let $L \subseteq \Sigma^*$ be a language. We say that M *accepts/recognizes* L if

$$L = \{w \in \Sigma^* \mid M \text{ accepts } w\}.$$

Let $L \subseteq \Sigma^*$ be a language. We say L is *regular* if $\exists$ a DFA M s.t.

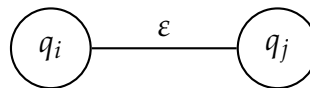
$$L = L(M).$$

Example 1.7. $A = \{10, 001\}$

$$A^* = \{\epsilon, 10, 001, 10001, 1010, 00110, \dots\}.$$

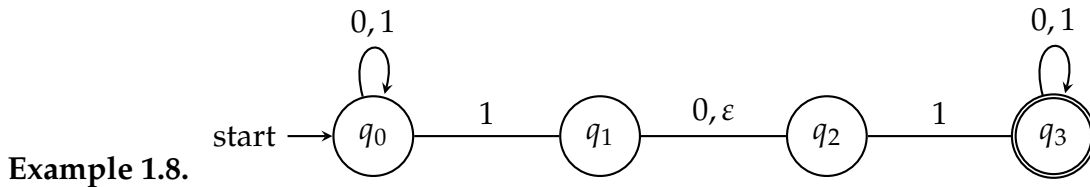
1.3 Non-Determinism

- From a state q , on an input symbol a , the automaton can go to multiple states (k different states, $k \geq 0$).
- Computation happens simultaneously along each of these paths.
- Has ϵ -transitions if there is an ϵ -transition



then the automaton moves to state q_j from q_i without reading the next input bit.

- An input is accepted if there is some computation path that leads to an accept state.



$$(q_0, 1) \rightarrow \{q_0, q_1\}$$

$$(q_1, 1) \rightarrow \{\}$$

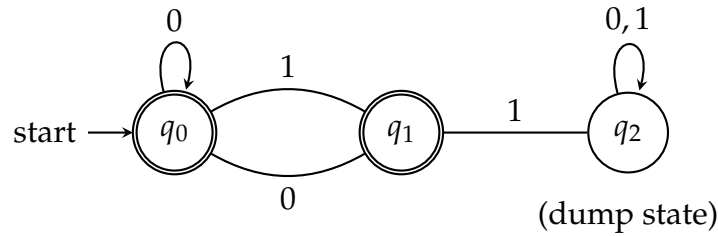
$$(q_1, 0) \rightarrow \{q_2\}$$

$$(q_1, \varepsilon) \rightarrow \{q_2\}$$

Note: The transitions are labelled with symbols from the set $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$.

Example 1.9. $L = \{0^n 1^n \mid n \geq 0\}$ is not regular.

Example 1.10. $L = \{w \mid w \text{ doesn't contain } 11 \text{ as a substring}\}$.

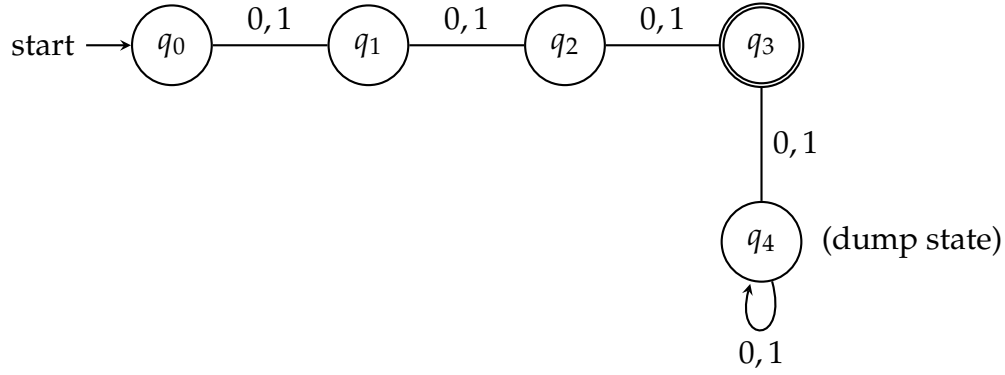


q_2 corresponds to the set of all strings that contain 11 as a substring. q_0 corr. to the set of strings that don't end with a 1. q_1 corr. to the set of all strings that don't contain 11 and end with 1.

$$M = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_0, q_1\}).$$

Remark 1.11. A *dump state* is a state from where the automaton cannot reach an accept state.

Example 1.12. $L = \{w \in \{0, 1\}^* \mid |w| = 3\}$.



1.3.1 Regular Operations

Let A, B be two languages over Σ .

1. **Union:** $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$
2. **Concatenation:** $A \cdot B = \{xy \mid x \in A, y \in B\}$
3. **Star:** $A^* = \{x_1 \cdots x_k \mid k \geq 0, x_i \in A \forall i\}$

1.4 Nondeterministic Finite Automata

Definition 1.13 (NFA). A *nondeterministic finite automaton* (NFA) is the five-tuple $N = (Q, \Sigma, \delta, q_0, F)$ with Q, Σ, q_0, F defined the same as for a DFA, but δ is different: instead of $\delta : Q \times \Sigma \rightarrow Q$, we have the transition function

$$\delta : Q \times \Sigma_\epsilon \rightarrow 2^Q, \quad [\Sigma_\epsilon = \Sigma \cup \{\epsilon\}].$$

Definition 1.14. We say that N *accepts* an input $w = a_1 \cdots a_n$ if we can write w as $w = b_1 \cdots b_m$ where $b_i \in \Sigma_\epsilon$ and $\exists$ a sequence of states $r_0, \dots, r_m$ (not necessarily distinct) such that

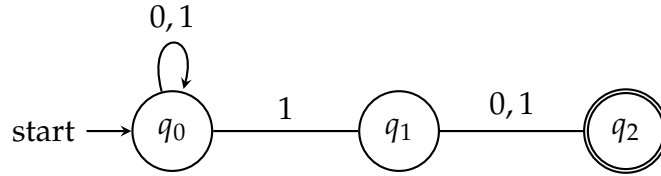
i) $r_0 = q_0$ (initial condition)

ii) $r_i \in \delta(r_{i-1}, b_i) \quad \forall i = 1, \dots, m$ (transition condition)

iii) $r_m \in F$. (accept condition)

We similarly define $L(N) := \{w \in \Sigma^* \mid N \text{ accepts } w\}$.

Example 1.15. $L_3 = \{w \in \{0,1\}^* \mid 3^{\text{rd}} \text{ last digit of } w \text{ is } 1\}$.



This choice (whether to stay at q_0 or move to q_1) is made by (nondeterministically) guessing if the digit read is the 3rd last digit of w .

Similarly we can define an NFA for

$$L_k = \{w \in \{0,1\}^* \mid k^{\text{th}} \text{ last digit of } w \text{ is } 1\}.$$

Consider an input $w = 010110$.

Set of states the automaton is in	Symbol read
$\{q_0\}$	0
$\{q_0\}$	1
$\{q_0, q_1, q_2\}$	0
$\{q_0, q_2\}$	1
$\{q_0, q_1, q_2, q_3\}$	1
$\{q_0, q_1, q_2, q_3\}$	0
$\{q_0, q_2, q_3\}$	Input accepted

1.4.1 Difference between DFA and NFA

DFA	NFA
$(q, a) \rightarrow$ single state	$(q, a) \rightarrow$ multiple states
single computation	multiple computation
no ε -transitions	have ε -transitions
accept if the computation ends at an accept state	accept if one of the computation paths ends at an accept state

1.5 Equivalence of NFA and DFA

Every DFA is also an NFA.

Theorem 1.16. *Let $N = (Q, \Sigma, \delta, q_0, F)$ be an NFA. There exists a DFA $D = (Q', \Sigma, \delta', q'_0, F')$ such that $L(N) = L(D)$.*

Proof (construction of D). • $Q' = 2^Q$.

- Let $A \subseteq Q$, then $\delta'(A, a) = \bigcup_{r \in A} \delta(r, a)$.
- $q'_0 = \{q_0\}$.
- $F' = \{A \subseteq Q \mid A \cap F \neq \emptyset\}$.

What about ε -transitions?

Let $R \subseteq Q$. Define the ε -closure of R as $E(R)$:

$$E(R) = \bigcup_{r \in R} \{t \mid t \text{ is reachable from } r \text{ using 0 or more } \varepsilon\text{-transitions}\}.$$

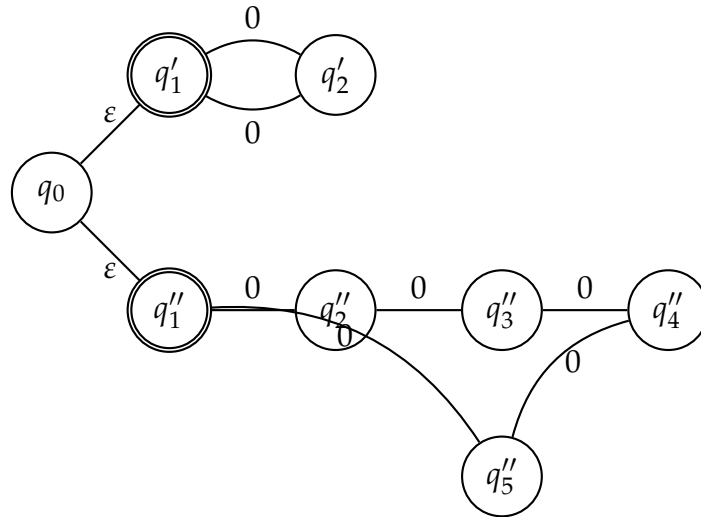
We define δ' to accommodate ε -transitions:

$$\delta'(A, a) = \bigcup_{r \in A} E(\delta(r, a)).$$

(Proof continued — refer to Hopcroft / Sipser / Kozen.) □

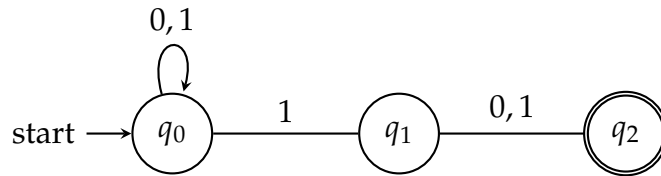
Example 1.17. $L_2 = \{w \in \{0, 1\}^* \mid |w| \text{ is div. by 2 or 5}\}$.

Solution.

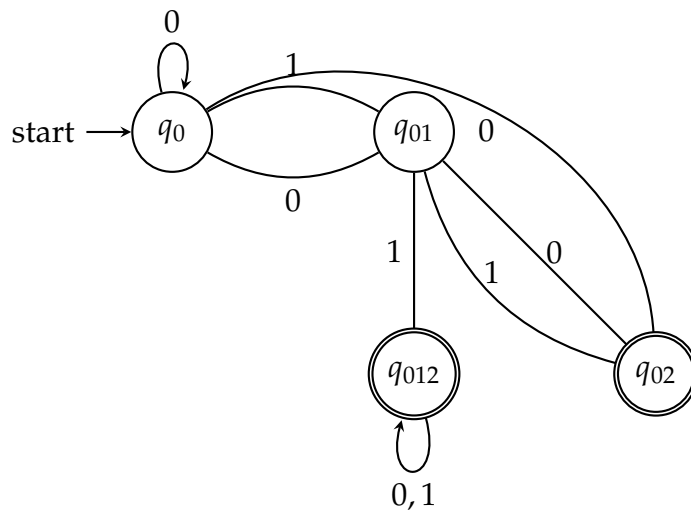


q_0 moves via ε into a 2-cycle (tracking length mod 2, accepting at the start of the cycle) and, separately, via ε into a 5-cycle (tracking length mod 5, accepting at the start of that cycle).

Example 1.18. $L = \{w \mid 2^{\text{nd}} \text{ last symbol in } w \text{ is } 1\}$.



Construction of DFA for this (subset construction):



$$\delta(q_0, 0) = q_0, \quad \delta(q_0, 1) = q_{01}, \quad \delta(q_{01}, 0) = q_{02}, \quad \delta(q_{01}, 1) = q_{012},$$

$$\delta(q_{02}, 0) = q_0, \quad \delta(q_{02}, 1) = q_{01}, \quad \delta(q_{012}, 0) = q_{012}, \quad \delta(q_{012}, 1) = q_{012},$$

$$F = \{q_{02}, q_{012}\}.$$

Remark 1.19. States that are not reachable from the start state are known as *dead states*.

1.6 Closure under Union, Concatenation, and Star

Theorem 1.20. If L_1 and L_2 are regular, then $L_1 \cup L_2$, $L_1 L_2$, and L_1^* are also regular.

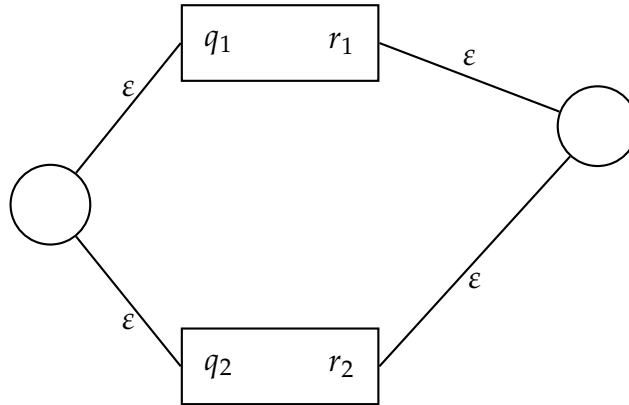
Remark 1.21. For a regular language, we can assume that there is an NFA accepting it with a unique accept state (just use ϵ -transitions).

Proof. Let N_1, N_2 be NFAs. Assume the following setup:

Language	NFA	start state	end state
L_1	N_1	q_1	r_1
L_2	N_2	q_2	r_2

with $L_1 = L(N_1)$ and $L_2 = L(N_2)$.

For $L_1 \cup L_2$:



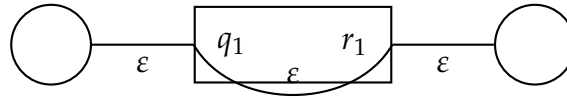
So every string that is accepted by L_1 or L_2 is accepted for this setup as well.

For $L_1 L_2$:



NFA for $L_1 \cdot L_2$.

For L_1^* :



NFA for L_1^* .



1.7 Regular Expressions

- Algebraic representation of languages.
- Applications in text editors to find strings, compiler design.

Egs. of regular expressions:

Regular Exp ⁿ	Languages
0	{0}
ϵ	{ ϵ }
$0 \cup 01$	{0, 01}
1^*	{ $\epsilon, 1, 11, \dots$ }
$(0 \cup 01) \cdot 1^*$	{0, 01, 011, 0111, ...}

Definition 1.22. R is said to be a *regular expression* (RE) if R has one of the following forms:

Reg. Exp.	Language	Comment
ϕ	{ }	
ϵ	{ ϵ }	
a	{ a }	$a \in \Sigma$
$R_1 \cup R_2$	$L(R_1) \cup L(R_2)$	
$R_1 \cdot R_2$	$L(R_1) \cdot L(R_2)$	
R_1^*	$L(R_1)^*$	
(R_1)	$L(R_1)$	

Remark 1.23. For every regular expression R , there is a unique language $L(R)$ corresponding to it.

- We often replace $\cup$ with the $+$ symbol and drop the $\cdot$ symbol.
- Precedence of the symbols/operators (High to Low): $()$, $*$, $\cdot$, $\cup$.
- $\phi^* = \{\epsilon\}$.

More Examples (RE $\rightarrow$ language):

Reg. Exp ⁿ	Language
01	$\{01\}$
$01 + 1$	$\{01, 1\}$
$(01 + \epsilon) \cdot 1$	$\{011, 01\}$
$(0 + 10)^* \cdot (\epsilon + 1)$	$\{\epsilon, 0, 1, 00, 01, 10, 000, 001, 101, 0000, \dots\}$

Example 1.24. (Language $\rightarrow$ RE)

- (i) $\{w \mid w \text{ has a single } 1\}$: $R = 0^* \cdot 1 \cdot 0^*$.
- (ii) $\{w \mid w \text{ has at most one } 1\}$: $R = 0^* \{\epsilon, 1\} 0^* = 0^* + 0^* 1 0^*$.
- (iii) $\{w \mid |w| \equiv 0 \pmod{3}\}$:

$$R = \{000, 001, \dots, 110, 111\}^* = ((0 + 1)(0 + 1)(0 + 1))^*.$$

- (iv) $\{w \mid w \text{ has a } 1 \text{ at every odd position and } |w| \text{ is odd}\}$:

$$R = \{10, 11\}^* \{1\} = (1(0 + 1))^* \cdot 1.$$

1.7.1 Properties of RE

1) (a) $R_1 + (R_2 + R_3) = (R_1 + R_2) + R_3$

(b) $R_1(R_2 R_3) = (R_1 R_2) R_3$

2) (a) $(R_1 + R_2) \cdot R_3 = R_1 \cdot R_3 + R_2 \cdot R_3$

(b) $R_1 \cdot (R_2 + R_3) = R_1 R_2 + R_1 R_3$

3) $R_1 + R_2 = R_2 + R_1$ (only $+$ is commutative)

4) (a) $R_1 + \phi = R_1$

(b) $R_1 \varepsilon = R_1$

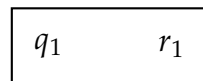
5) $(R_1^*)^* = R_1^*$.

Theorem 1.25. A language L is regular iff there is a regular expⁿ R s.t. $L = L(R)$.

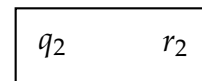
Proof (if part). This proof is based on case analysis.

RE	NFA
ϕ	single non-accepting state
ε	single start = accept state
a	two states, start $\xrightarrow{a}$ accept

$R_1 + R_2$: Suppose NFA of R_1 & R_2 resp. are

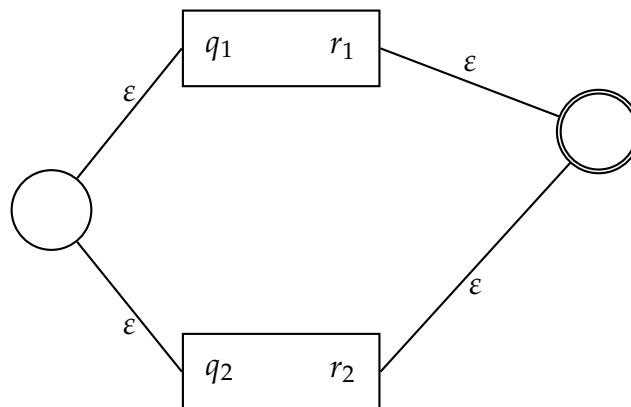


N_1 for $L(R_1)$

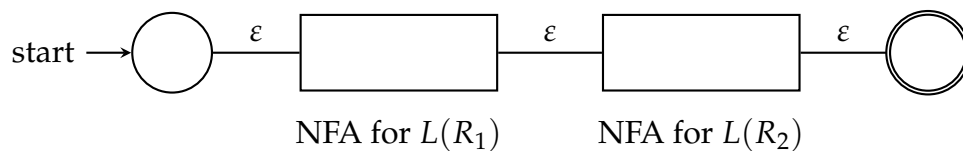


N_2 for $L(R_2)$

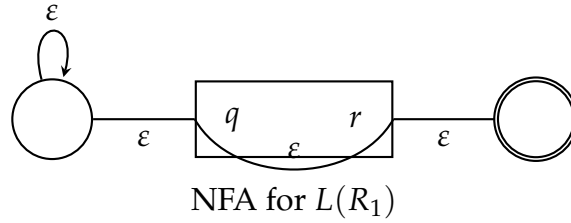
$\therefore$ NFA for $L(R_1) \cup L(R_2)$ or $R_1 + R_2$ is



$R_1 R_2$:



R_1^* :



So given an RE, we can explicitly construct the NFA. □

1.8 Converting a DFA to a Regular Expression

A GNFA is an NFA with the transitions being labelled by regular expressions.

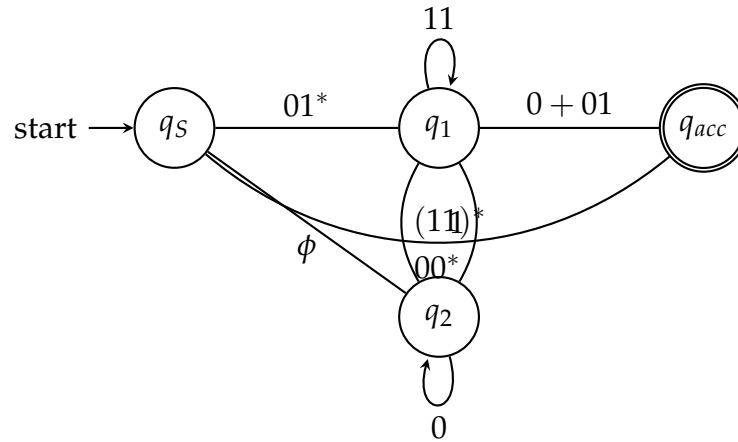
Definition 1.26. A GNFA is said to *accept* a string w if w can be written as $w = w_1 \cdots w_k$ (concatenation of k strings) and $\exists$ states $q_0, q_1, \dots, q_k$ in the GNFA such that

- i) q_0 is the start state.
- ii) q_k is an accept state.
- iii) $\forall i$, if R_i is the label on the transition q_{i-1} to q_i , then $w_i \in L(R_i)$.

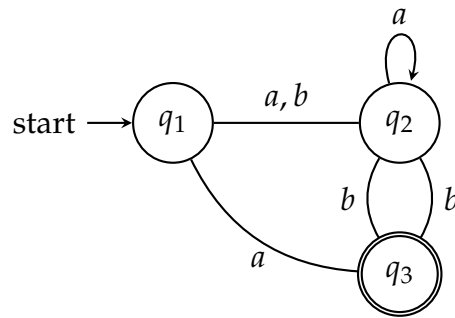
Without loss of generality, we may assume the following:

1. GNFA has a unique accept state.
2. There are no incoming transitions to the start state and no outgoing transitions from the accept state.
3. There are transitions from the start state to every other state, and from every state to the unique accept state.
4. There is a transition between every pair of states that are not the start/accept state.

Example 1.27. Here is an example of a generalized NFA (GNFA), i.e. an NFA with regular-expression-labelled transitions.



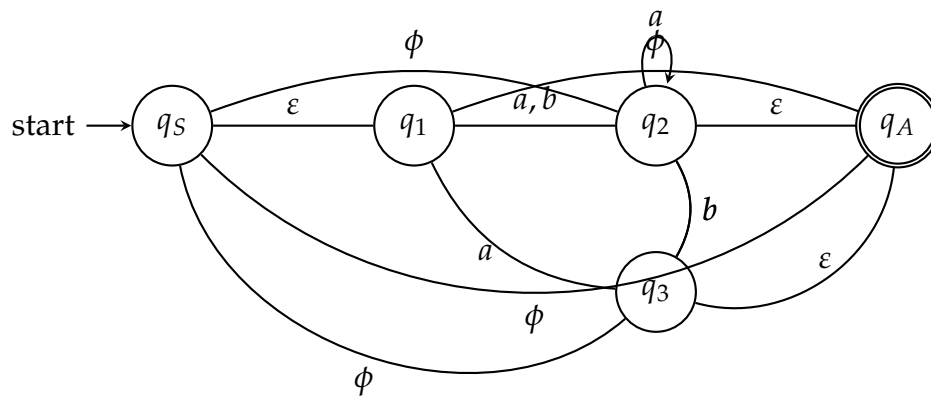
1.8.1 A Motivating Example



Algorithm for DFA $\rightarrow$ RE:

1. Convert the DFA to a GNFA in the appropriate form.
2. Removing a state.

Step 1 (GNFA form):



Step 2 (removing a state – general picture): removing q_{rem} from $q_A \xrightarrow{R_0} q_B$, $q_A \xrightarrow{R_1} q_{\text{rem}}$, $q_{\text{rem}} \xrightarrow{R_2} q_{\text{rem}}$ (self-loop), $q_{\text{rem}} \xrightarrow{R_3} q_B$ gives

$$q_A \xrightarrow{R_0 + R_1 R_2^* R_3} q_B.$$

Removing q_1 : the states q_S, q_2, q_A, q_3 remain, with (among others) the transition

$$q_S \xrightarrow{\phi + \varepsilon \cdot \phi^* (a+b)} q_2, \quad q_S \xrightarrow{\phi + \varepsilon \cdot \phi^* \cdot \varepsilon} q_A, \quad q_S \xrightarrow{\phi + \varepsilon \cdot \phi^* \cdot \phi} q_3, \quad \text{etc.}$$

Removing q_2 : with q_S, q_A, q_3 remaining,

$$q_S \xrightarrow{\phi + (a+b) \cdot a^* \cdot \phi} q_3 = (a+b)a^*b, \quad q_3 \xrightarrow{\phi + (b+a(a+b)) \cdot a^* \cdot b} q_3 = (b+a(a+b))a^*b,$$

$$q_3 \xrightarrow{(\varepsilon+a) + (b+a(a+b))a^* \cdot \phi} q_A = \varepsilon + a.$$

Removing q_3 : with q_S, q_A remaining,

$$q_S \xrightarrow{\varepsilon + (a+b)a^*b \cdot ((b+a(a+b))a^*b)^* (\varepsilon+a)} q_A.$$

Required RE:

$$R = \varepsilon + (a+b)a^*b((b+a(a+b))a^*b)^*(\varepsilon+a).$$

1.9 More Closure Properties of Regular Languages

Complement

Complement of L : $\bar{L} = \Sigma^* \setminus L$.

Proposition 1.28. *If L is regular, then $\bar{L}$ is also regular.*

Proof. Let $D = (Q, \Sigma, \delta, q_0, F)$ be some DFA for L . We construct a DFA D' for $\bar{L}$ where

$$D' = (Q, \Sigma, \delta, q_0, Q \setminus F).$$

□

Intersection

Proposition 1.29. *If L_1 and L_2 are regular, then $L_1 \cap L_2$ is also regular.*

Proof. From De Morgan's law, $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$.

Now L_1, L_2 are regular $\implies \overline{L_1} \cup \overline{L_2}$ is regular $\implies L_1 \cap L_2$ is regular. □

Set Difference

Proposition 1.30. *If L_1 and L_2 are regular, then $L_1 \setminus L_2$ is also regular.*

Proof. L_1, L_2 regular $\implies \overline{L_1}, \overline{L_2}$ regular $\implies L_1 \cap \overline{L_2}$ regular $\implies L_1 \setminus L_2$ regular. □

1.10 Reversal of a Language

Let $w = a_1a_2 \cdots a_n$ be a string, then

$$\text{rev}(w) = a_na_{n-1} \cdots a_2a_1.$$

For a language $L \subseteq \Sigma^*$,

$$\text{rev}(L) := \{w \in \Sigma^* \mid \text{rev}(w) \in L\}.$$

Proposition 1.31. *If L is regular, then $\text{rev}(L)$ is also regular.*

Proof. Let $D = (Q, \Sigma, \delta, q_0, F)$ be a DFA for L . Now define an NFA D' as the following.

$$D' = (Q', \Sigma, \delta', q'_0, F'), \quad \text{where } Q' = Q \cup \{q'_0\}$$

$$\delta'(q, a) = \{r \mid \delta(r, a) = q\}, \quad \delta'(q'_0, \varepsilon) = \{f \mid f \in F\}, \quad F' = \{q'_0\}.$$

Now $L(D') = \text{rev}(L)$.

$\therefore \text{rev}(L)$ is also regular. □

1.11 Homomorphisms

Definition 1.32 (Homomorphism). Let Σ, Γ be two alphabets. A *homomorphism* $h : \Sigma \rightarrow \Gamma^*$ is a map such that if $w = a_1 a_2 \cdots a_n \in \Sigma^*$, then

$$h(w) = h(a_1)h(a_2) \cdots h(a_n).$$

Also, $h(L) := \{h(w) \mid w \in L\}$ where $L \subseteq \Sigma^*$.

Proposition 1.33. *If L is regular, then $h(L)$ is regular.*

1.12 Inverse Homomorphism

Let $h : \Sigma \rightarrow \Gamma^*$ be a homomorphism. For $L \subseteq \Gamma^*$, define

$$h^{-1}(L) = \{w \in \Sigma^* \mid h(w) \in L\}.$$

Proposition 1.34. *If L is regular, then $h^{-1}(L)$ is regular.*

Proof. Proof of the above two propositions are via the same method: as L is regular, we get a regular expression for L , and applying h to each symbol in that RE, we get an RE for $h(L)$, thus it is regular. Same for $h^{-1}(L)$. $\square$

NON-REGULAR LANGUAGES

There exist non-regular languages. Non-regular languages require us to maintain some count. The following theorem helps us to sometimes check if a language is non-regular.

Lemma 1.35 (The Pumping Lemma for Regular Languages). *Let L be a regular language.*

Then $\exists p \geq 0$ s.t. $\forall w \in L$ with $|w| \geq p$, $\exists$ a partition of w as $w = xyz$ such that

(i) $|xy| \leq p$

(ii) $y \neq \varepsilon$

(iii) $\forall k \geq 0, xy^kz \in L$.

Corollary 1.36 (Contrapositive of the Pumping Lemma). $\forall p \geq 0, \exists w \in L$ with $|w| \geq p$ such that $\forall$ partitions $w = xyz$ with $|xy| \leq p, y \neq \varepsilon$, and $\exists k \geq 0$ s.t. $xy^kz \notin L$, then L is not regular.

Example 1.37. $L_1 = \{0^n 1^n \mid n \geq 0\}$ is not regular.

Take any $p \geq 0$. Choose $w = 0^p 1^p$, also take any partition of $w = xyz$ with $|xy| \leq p$ and $y \neq \varepsilon$.

$\because |xy| \leq p \implies 1 \notin x, 1 \notin y$, i.e., x, y are made up of 0's only.

Choose $k = 0$, then $xy^0z = 0^l 1^p$ where $l = p - |y| \neq p$.

$\therefore xy^0z \notin L_1 \implies L_1$ is not regular. ■

Example 1.38. $L_2 = \{a^l b^m c^n \mid l + m \leq n\}$ is not regular. $[\Sigma = \{a, b, c\}]$

Take $p \geq 0$. Choose $w = a^p b^p c^{2p}$. Given a partition $w = xyz$ s.t. $|xy| \leq p$ and $y \neq \varepsilon$, we have $y = a^t$ for some $t > 0$.

Choose $k = 2$: $xy^2z = a^{p+t}b^pc^{2p}$.

Now $p + t + p \not\leq 2p \implies xy^2z \notin L_2$

$\implies L_2$ is not regular.

Example 1.39. $L_3 = \{w \in \{0,1\}^* \mid \#0 \text{ in } w = \#1 \text{ in } w\}$ is not regular.

Notice that $L(0^*1^*)$ is regular. Suppose for the sake of contradiction, L_3 is regular. Then

$$L(0^*1^*) \cap L_3 \text{ is regular}$$

$$\implies L_1 = \{0^n1^n \mid n \geq 0\} \text{ is regular.} \implies \Leftarrow$$

$\therefore L_3$ is not regular. ■

Example 1.40. $L_4 = \{0^p \mid p \in \mathbb{P}\}$ (the set of primes) is not regular. $[\Sigma = \{0\}]$

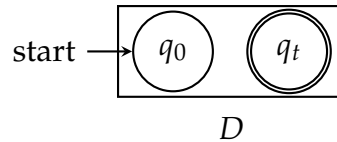
Given $q \geq 0$, choose $w = 0^r$ where $r \in \mathbb{P}, r \geq 2q$. Now given a partition of $w = xyz$ where $|xy| \leq q$ & $|y| = t > 0$, choose $i = r + 1$, then

$$xy^iz = 0^{r(1+t)} \notin L_4.$$

$\therefore L_4$ is not regular. ■

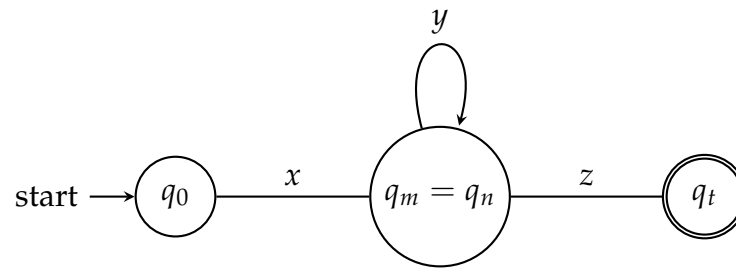
Proof of the Pumping Lemma for Regular Languages

Proof. Let $D = (Q, \Sigma, \delta, q_0, F)$ be a DFA for L . Set $p = |Q|$.



Given $w \in L$ such that $|w| \geq p$, $\exists$ a sequence of states $q_0, \dots, q_t$ such that $t \geq p$ and $q_t \in F$.

By the Pigeonhole Principle, $\exists m, n \geq 0$ such that $0 \leq m < n \leq t$ & $q_m = q_n$.



Set x = the string from q_0 to q_m , and let m be the first time there is such a repetition. Set y = string on the loop (first loop), z = remaining string.

Now $\forall i, xy^iz \in L$.

(*) all the $|w| \geq p, |xy| \leq p, y \neq \varepsilon$ take care of themselves from $p = |Q|$, y pumped, etc.

■

□

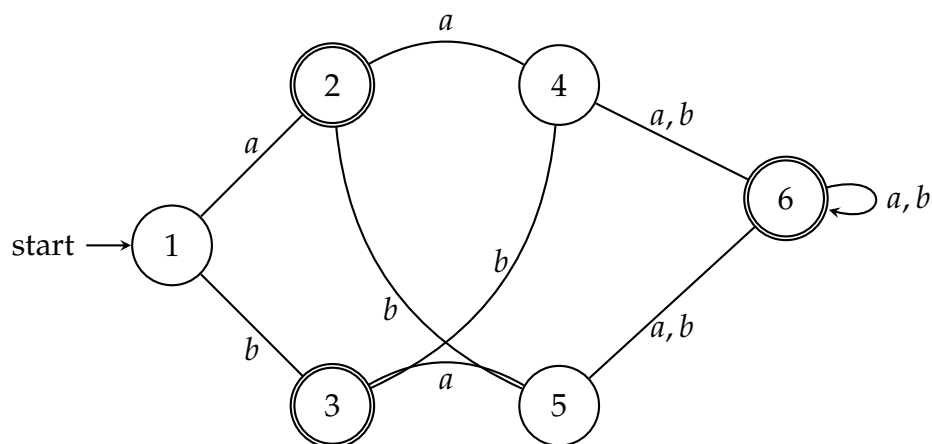
DFA MINIMIZATION

DFA minimization is an algorithm that produces a minimal DFA from a given DFA.

Algorithm

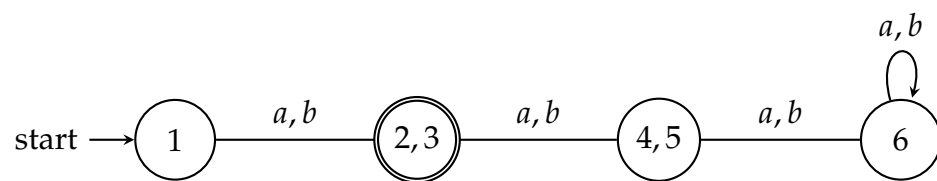
Input: $D = (Q, \Sigma, \delta, q_0, F)$.

1. Construct a table of all $\{p, q\}$ pairs where $p, q \in Q$.
2. Mark a pair $\{p, q\}$ if $p \in F$ and $q \notin F$, or vice versa.
3. Repeat the following until no more pairs can be marked: mark $\{p, q\}$ if $\{\delta(p, a), \delta(q, a)\}$ is marked for some $a \in \Sigma$.
4. Two or more states are equivalent if they are not marked.



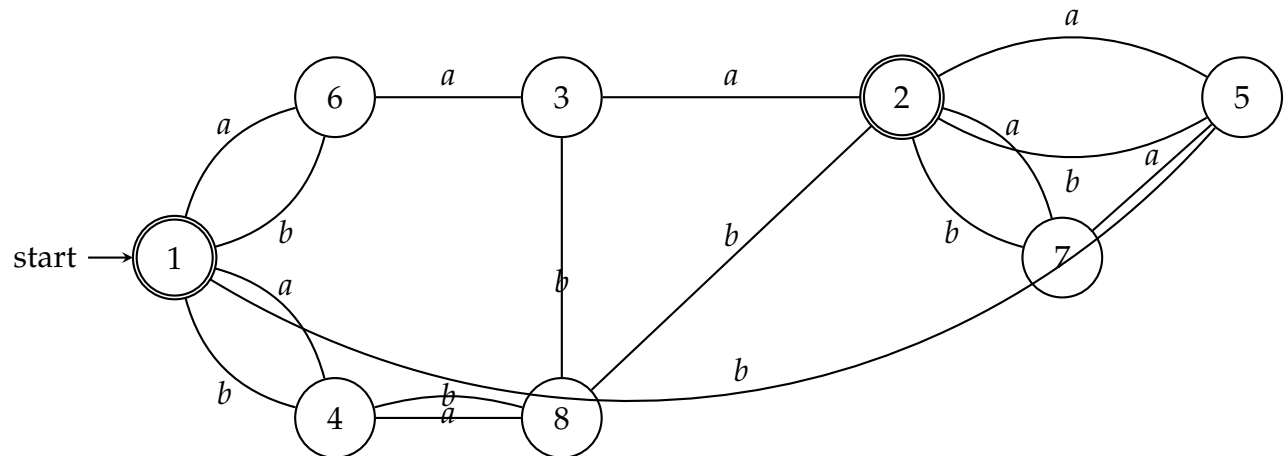
Example 1.41.

1					
×	2				
×		3			
×	×	×	4		
×	×	×		5	
×	×	×	×	×	6



This is the minimized DFA for the given DFA above.

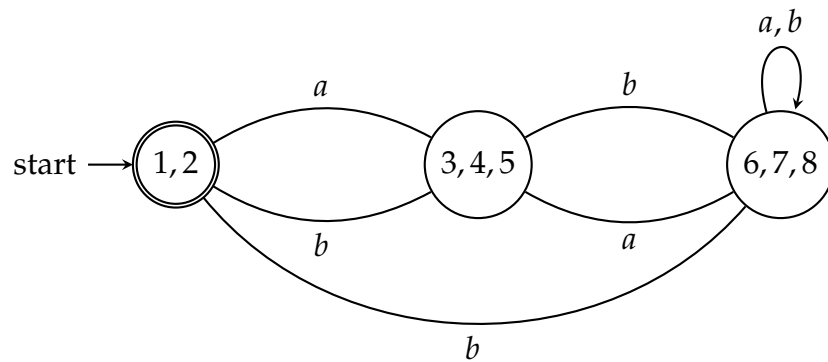
Example 1.42.



1							
×	2						
×		3					
×	×			4			
×	×				5		
×	×	×	×	×	×	6	
×	×	×	×	×	×		7
×	×	×	×	×			8

$$\{1 \approx 2\}, \quad \{3 \approx 4\}, \quad \{3 \approx 5\}, \quad \{6 \approx 7\}, \quad \{6 \approx 8\}, \quad \{7 \approx 8\}.$$

So the required minimized DFA is



CHAPTER 2

CONTEXT-FREE GRAMMARS AND PUSHDOWN AUTOMATA

2.1 Context-Free Grammars

Definition 2.1 (Context-Free Grammar). A CFG is a quadruple $G = (N, \Sigma, P, S)$

- N – finite set (nonterminal symbols)
- Σ – finite set, $N \cap \Sigma = \emptyset$ (terminal symbols)
- $P \subseteq N \times (N \cup \Sigma)^*$ finite (productions)
- $S \in N$ (start symbol)

We say G accepts the language

$$L(G) := \left\{ x \in \Sigma^* \mid S \xrightarrow[G]{*} x \right\}.$$

This is called a *context-free language* (CFL).

Example 2.2. $\{0^n 1^n \mid n \geq 0\}$ is a CFL.

Proof. Consider $S \rightarrow 0S1 \mid \varepsilon$.

Example 2.3. Set of all balanced parentheses is a CFL.

Proof. $L = \{x \in \{[,]\}^* \mid x \text{ is balanced}\}$.

Observation 2.4. $x \in \{[,]\}^*$ is balanced iff

(1) $L(x) = R(x)$, and

(2) $L(y) \geq R(y) \quad \forall \text{ prefix } y \text{ of } x$.

(Here $L(x)$, $R(x)$ denote the number of left and right brackets in x respectively.)

Consider the CFG

$$G : \quad S \rightarrow [S] \mid SS \mid \varepsilon.$$

(Why?) We use induction. Both directions:

(i) If $S \xrightarrow[G]{*} x$ then x balanced.

(ii) If x is balanced, then $S \xrightarrow[G]{*} x$ (just case analysis). ■

Example 2.5. $\{0^n 1^m \mid n > 0, n \leq m \leq 2n\}$.

$$S \rightarrow ASB \mid \varepsilon, \quad A \rightarrow 0, \quad B \rightarrow 1 \mid 11.$$

Example 2.6. $\{x \in \{0, 1\}^* \mid \#_0(x) = \#_1(x)\}$.

$$S \rightarrow 0S1S \mid 1S0S \mid \varepsilon.$$

All Regular Languages are CFLs

Proposition 2.7. *Every regular language is a CFL.*

Proof. Let L be a regular language with the DFA $M = (Q, \Sigma, \delta, q_0, F)$. We construct a CFG G for L :

$$N = \{R_i \mid q_i \in Q\}, \quad \Sigma = \Sigma,$$

$$P = \{(R_i, aR_j) \mid \text{if } \delta(q_i, a) = q_j, a \in \Sigma\} \cup \{(R_i, \varepsilon) \mid q_i \in F\},$$

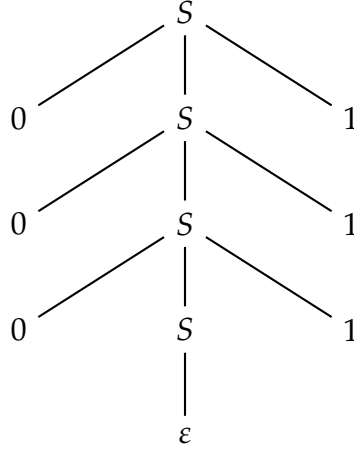
$$S = R_0.$$

Then we have $L(G) = L$. □

2.2 Parse Trees, Derivation & Ambiguity

Definition 2.8 (Parse Tree). For a CFG G , a *parse tree* of $w \in L(G)$ w.r.t. G is a rooted ordered tree corresponding to a derivation of w by G .

Example 2.9. $L = \{0^n 1^n \mid n \geq 0\}$, $S \rightarrow 0S1 \mid \varepsilon$, $w = 0^3 1^3$.



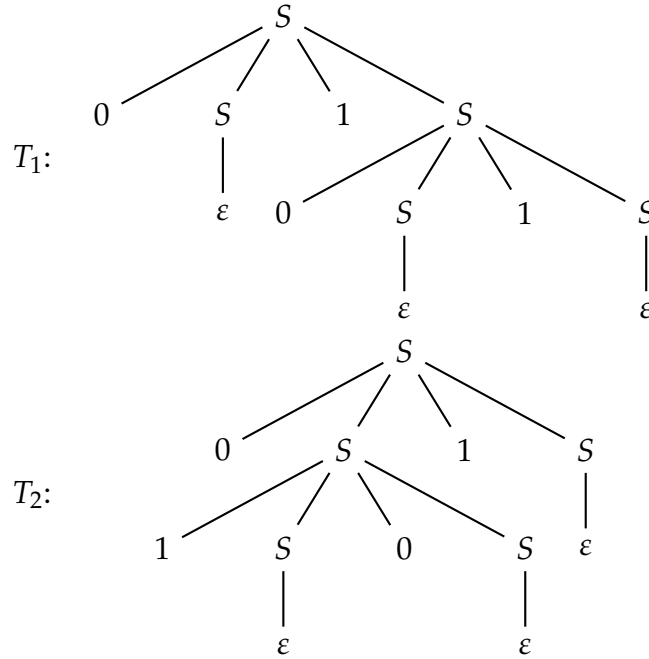
Remark 2.10 (Some Properties). (1) All internal nodes are variables.

(2) All leaf nodes are from $\Sigma \cup \{\varepsilon\}$.

(3) If ε is a leaf node, then it is the only child.

(4) Concatenation of all symbols from left to right (all leaf nodes) gives the derivation of w .

Parse trees may not be unique. Take $G_2 : S \rightarrow 0S1S \mid 1S0S \mid \varepsilon$, $w = 0101$. Two parse trees for w w.r.t. G_2 are:



Corresponding derivations (replacing the *leftmost* variable first at each step – called *leftmost derivation*):

$$T_1 : S \Rightarrow 0S1S \Rightarrow 01S \Rightarrow 010S1S \Rightarrow 0101S \Rightarrow 0101.$$

$$T_2 : S \Rightarrow 0S1S \Rightarrow 0S1 \Rightarrow 01S0S1 \Rightarrow 010S1 \Rightarrow 0101.$$

Definition 2.11 (Ambiguous string). A string w is called *ambiguous* if it has at least two different leftmost derivations.

Lemma 2.12. w is ambiguous iff it has two distinct parse trees.

Definition 2.13. A CFG G is *ambiguous* if $\exists w \in L(G)$ which is ambiguous.

A CFL L is *ambiguous* if $\forall$ CFG G with $L(G) = L$, G is ambiguous.

2.3 Normal Forms

Definition 2.14 (Chomsky Normal Form). A CFG $G = (N, \Sigma, P, S)$ is said to be in *Chomsky Normal Form* (CNF) if

- (i) Every production rule is of the form

$$A \rightarrow BC \quad (B, C \in N), \quad A \rightarrow a \quad (a \in \Sigma).$$

- (ii) S doesn't appear on the r.h.s. of any rule.

- (iii) The rule $S \rightarrow \varepsilon$ may be present, depending on whether $\varepsilon \in L(G)$ or not.

Theorem 2.15. *Every CFL has a CNF grammar.*

Proof. Proof via an algorithm. Let $G = (N, \Sigma, P, S)$ be a CFG for the CFL L . We shall convert G to a CNF grammar.

1. If S appears on the r.h.s. of any rule, add a new start variable S_0 and add the rule $S_0 \rightarrow S$.
2. (Removing ε -transitions) For a rule $A \rightarrow \varepsilon$, remove this rule, and for every occurrence of A on the r.h.s. (of any other rule), remove that occurrence and add a new rule. E.g.

$$A \rightarrow \varepsilon, \quad B \rightarrow uAv, \quad C \rightarrow u_1Au_2Au_3$$

$$\implies B \rightarrow uAv \mid uv, \quad C \rightarrow u_1Au_2Au_3 \mid u_1u_2Au_3 \mid u_1Au_2u_3 \mid u_1u_2u_3.$$

3. (Removing Unit Rules)

$$A \rightarrow B, \quad B \rightarrow u \quad \implies \quad A \rightarrow u, \quad B \rightarrow u.$$

4. (Shortening R.H.S.) For $A \rightarrow u_1u_2 \cdots u_k$ ($k \geq 3$), add a variable A_1 :

$$A_1 \rightarrow u_1u_2, \quad A \rightarrow A_1u_3 \cdots u_k.$$

5. (Add Variables) For $A \rightarrow uv$ with $u \in \Sigma$:

$$U_1 \rightarrow u, \quad A \rightarrow U_1 v.$$

Example. Consider

$$S \rightarrow ASB, \quad A \rightarrow aASA \mid a \mid \varepsilon, \quad B \rightarrow SbS \mid A \mid bb.$$

Step 1:

$$S_0 \rightarrow S, \quad S \rightarrow ASB, \quad A \rightarrow aASA \mid a \mid \varepsilon, \quad B \rightarrow SbS \mid A \mid bb.$$

Step 2 (remove $A \rightarrow \varepsilon$):

$$S_0 \rightarrow S, \quad S \rightarrow ASB \mid SB, \quad A \rightarrow aASA \mid aSA \mid aAS \mid a \mid aS,$$

$$B \rightarrow SbS \mid A \mid bb \mid \varepsilon.$$

Step 2 continued (remove $B \rightarrow \varepsilon$):

$$S_0 \rightarrow S, \quad S \rightarrow ASB \mid AS \mid SB, \quad A \rightarrow aASA \mid aSA \mid aAS \mid a \mid aS,$$

$$B \rightarrow SbS \mid A \mid bb.$$

□

2.4 Pumping Lemma for CFLs

Lemma 2.16 (Pumping Lemma for CFLs). *For a CFL L , $\exists k \in \mathbb{N}$ such that $\forall z \in L$ with $|z| \geq k$, z can be partitioned like $z = uvwxy$ such that*

1. $|vwx| \leq k$
2. $vx \neq \varepsilon$
3. $uv^iwx^iy \in L$ for all $i \geq 0$.

Proof. Consider a CFL L and let $G = (N, \Sigma, P, S)$ be the CNF grammar for L . Consider $k = 2^{n+1}$ and $z \in L$ with $|z| \geq k$. Consider a parse tree of z w.r.t. G .

$$\because \text{number of terminals} \geq 2^{n+1} \implies \text{depth of this tree} \geq n+1, \quad n = |N|.$$

Now consider the longest path in this tree; of course $\exists$ at least one repetition of a non-terminal (by the Pigeonhole Principle).

Take the first terminal occurrence from the leaf to root direction, let that non-terminal be X . Suppose w is the string generated by the lower occurrence of X and vwx be the string generated by the upper occurrence of X ; of course $|vx| \neq 0$ as we are in CNF.

Define T = tree with upper X as root, t = tree with lower X as root.

Now if we remove t and replace it by a copy of T , we yield the string uv^2wx^2y . Doing it again yields uv^3wx^3y , and so on. Replacing T by t yields uv^0wx^0y .

Note that $|vwx| \leq k$, as if not, then the tree T would have $\geq n+1$ depth, hence PHP would give a contradiction. $\square$

2.5 Proving a Language is not a CFL

A Demon Game

To show a language L is not a CFL, one may use the following game (it may not always work).

1. Demon picks $k \in \mathbb{N}_0$.
2. You pick $z \in L$ of length $\geq k$.
3. Demon gives you a partition $z = uvwxy$ with $|vx| > 0$, $|vwx| \leq k$.
4. If you get $i \in \mathbb{N}_0$ such that $uv^iwx^iy \notin L$, you win.

Example 2.17. $L = \{0^n 1^n 0^n \mid n \geq 0\}$ is not a CFL.

Take $z = 0^k 1^k 0^k$. Let $z = uvwxy$ be a partition given to us (with k given).

If $vwx = 1^m$ for some $m \leq k$, then done.

Else $vwx = 0^a 1^b$ or $1^a 0^b$ with $a + b \leq k$. Now doing v^iwx^i , after some point, the balance between the number of 0's on the left and right doesn't exist. ■

Example 2.18. $L = \{ww \mid w \in \Sigma^*\}$ is not context-free. $[\Sigma = \{a, b\}]$

We won't use the pumping lemma directly. Observe that the class of CFLs is closed under union (and, as used below, under intersection with a regular language).

Let L be a CFL, then so is

$$L \cap a^* b^* a^* b^* = \{a^n b^m a^n b^m \mid m, n \in \mathbb{N}_0\} =: L'.$$

Now let $k \in \mathbb{N}_0$ and $z = a^k b^k a^k b^k$. Let a partition of z be given as $z = uvwxy$.

Case I: one of v, x contains both a and b . Then $uv^2wx^2y \notin a^* b^* a^* b^*$.

Case II: v, x are part of the same block $\implies uv^2wx^2y \notin L'$.

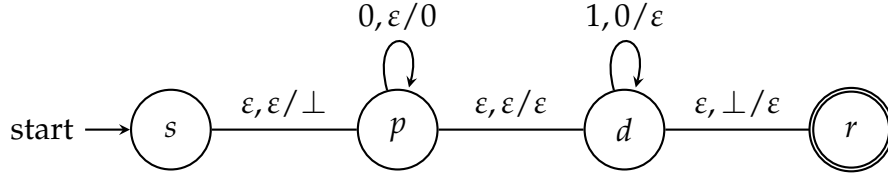
Case III: v, x are part of different blocks $\implies$ the blocks have to be adjacent, then also uv^2wx^2y creates a disbalance.

$$\implies L' \text{ is not a CFL. } \implies \Leftarrow$$

$\therefore L$ is not context-free. ■

2.6 Pushdown Automata

Example 2.19. $L = \{0^n 1^n \mid n \geq 0\}$, $\Sigma = \{0, 1\}$, $\Gamma = \{\perp, 0\}$.



2.7 NPDA and CFGs are Equivalent

Here is a proof (sketch) that the languages accepted by an NPDA are precisely CFLs, and all CFLs are accepted by an NPDA.

CFG → NPDA Conversion

Let $G = (N, \Sigma, P, S)$ be a grammar for the CFL L in Greibach Normal Form, i.e. every production rule is of the form

$$A \rightarrow cB_1 \cdots B_k; \quad A, B_i \in N, c \in \Sigma_\epsilon, k \geq 0.$$

Construct an NPDA M with one state as follows:

$$M = (\{q\}, \Sigma, N, \delta, q, S, \emptyset),$$

with $((q, c, A), (q, B_1 \cdots B_k)) \in \delta$ whenever $A \rightarrow cB_1 \cdots B_k$ is a production rule, and M accepts via emptying the stack.

The following lemma helps us to prove that $L(M) = L(G)$.

Lemma 2.20. *Let $z, y \in \Sigma^*$, $\gamma \in N^*$, $A \in N$. Then*

$$A \xrightarrow[G]{n} z\gamma \text{ via left most derivation} \quad \text{iff} \quad (q, zy, A) \xrightarrow[M]{n} (q, y, \gamma).$$

Theorem 2.21. $L(M) = L(G)$.

Proof.

$$\begin{aligned}
 x \in L(G) &\iff S \xrightarrow[G]{*} x \text{ via left most der.} \\
 &\iff (q, x, S) \xrightarrow[M]{*} (q, \varepsilon, \varepsilon) \\
 &\iff x \in L(M).
 \end{aligned}$$

□

NDPA $\rightarrow$ CFG

We shall first convert the NPDA to another equivalent NPDA with one state, and then convert it into a CFG.

Say we can simulate our NPDA M' with another NPDA $M = (\{q\}, \Sigma, \Gamma, \delta, q, \perp, \emptyset)$. Now construct the CFG $G = (\Gamma, \Sigma, P, \perp)$ as follows:

$$A \rightarrow cB_1 \cdots B_k \text{ in a production rule whenever } ((q, c, A), (q, B_1 \cdots B_k)) \in \delta.$$

The lemma proved before shows that $L(M) = L(G)$.

2.8 Closure Properties of CFLs

Union

L_1 – via a CFG $G_1 = (V_1, \Sigma, P_1, S_1)$, L_2 – via a CFG $G_2 = (V_2, \Sigma, P_2, S_2)$.

We shall construct a CFG $G = (V, \Sigma, P, S)$ s.t. $L(G) = L_1 \cup L_2$:

$$V = V_1 \cup V_2 \cup \{S\}, \quad P = P_1 \cup P_2 \cup \{S \rightarrow S_1 \mid S_2\}.$$

Concatenation

Just replace $S \rightarrow S_1 \mid S_2$ with $S \rightarrow S_1 S_2$ in the previous grammar. $L(G) = L_1 L_2$.

Star

In G_1 , add the production rule $S \rightarrow S_1 S$. $L(G_1) = L^*$.

Reversal

CFLs are closed under reversal. Whenever there is $A \rightarrow A_1 \cdots A_k$, replace it by $A \rightarrow A_k \cdots A_1$ to get G' .

$$L(G') = \text{rev}(L(G_1)).$$

CFLs are also closed under homomorphisms and inverse homomorphisms.

Intersection

Slightly different from the previous properties.

Proposition 2.22. *If L_1 is a CFL and L_2 is a regular language, then $L_1 \cap L_2$ is a CFL.*

Proof. Let $M_1 = (Q_1, \Sigma, \Gamma, \delta_1, q_1, \perp, F_1)$ be a PDA for L_1 and $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ be an NFA for L_2 . We look at the product PDA of M_1, M_2 . Let $M = (Q, \Sigma, \Gamma, \delta, q, \perp, F)$ be defined as

$$Q = Q_1 \times Q_2, \quad q = (q_1, q_2), \quad F = F_1 \times F_2.$$

□

CHAPTER 3

TURING MACHINES AND EFFECTIVE COMPUTABILITY

CHAPTER 4

SELECTED SOLUTIONS TO MISCELLANEOUS PROBLEMS (KOZEN)
